Applications Libraries

Libraries/nix/applications/filters.nix

```
{_, ...}: let
  __exports = {
    internal = filters;
    external.applicationFilters = filters;
  };

  inherit (_.applications.construction) mkFilters;
  inherit (_.applications.enums) categories channels families;
  inherit (_.attrsets.access) attrNames attrValues;
  inherit (_.attrsets.construction) genAttrs;
  inherit (_.attrsets.transformation) filterAttrs mapAttrs;
  inherit (_.lists.access) length;
  inherit (_.lists.predicates) isIn;
  inherit (_.lists.transformation) unique;
  inherit (_.lists.reduction) concatMap;
  inherit (_.lists.selection) filter;
  all = _.applications.registry;

  #╔═══════════════════════════════════════════════════════════╗
  #║ Checkers                                                  ║
  #╚═══════════════════════════════════════════════════════════╝
  withFlag = {
    field,
    set,
  }:
    filterAttrs
    (_: a: (a.${field} or false))
    set;

  withoutFlag = {
    field,
    set,
  }:
    filterAttrs
    (_: a: !(a.${field} or false))
    set;

  withEq = {
    field,
    default ? null,
    value,
    set,
  }:
    filterAttrs
    (_: a: (a.${field} or default) == value)
    set;

  withMember = {
    field,
    value,
    set,
  }:
    filterAttrs
    (_: a: isIn value (a.${field} or []))
    set;

  groupBy = {
    field,
    default ? "unknown",
    keys
```

```
  keys,
  set,
}:
  genAttrs keys (k:
    withEq {
      inherit field default set;
      value = k;
    });

groupByMember = {
  field,
  keys,
  set,
}:
  genAttrs keys (p:
    withMember {
      inherit field set;
      value = p;
    });

#╔═══════════════════════════════════════════════════════════╗
#║ Builders                                                  ║
#╚═══════════════════════════════════════════════════════════╝
mkEqQueries = {
  field,
  default ? false,
  set,
}: let
  keys = unique (
    filter (k: k != default) (
      map
      (a: a.${field} or default)
      (attrValues set)
    )
  );
in
  groupBy {inherit field default keys set;};

mkBoolQueries = {
  field,
  trueKey,
  falseKey,
  set,
}: {
  ${trueKey} = withFlag {inherit field set;};
  ${falseKey} = withoutFlag {inherit field set;};
};

mkMemberQueries = {
  field,
  set,
}: let
  keys = unique (
    concatMap
    (a: a.${field} or [])
    (attrValues set)
  );
in
  groupByMember {inherit field keys set;};

mkLengthQueries = {
  field,
  singleKey,
  multiKey,
  set,
}: {
```

```
  ${singleKey} =
    filterAttrs
    (_: a: length (a.${field} or []) == 1)
    set;
  ${multiKey} =
    filterAttrs
    (_: a: length (a.${field} or []) > 1)
    set;
};

mkScopeQueries = {
  set,
  field,
  singleKey,
  multiKey,
}:
  mkLengthQueries {inherit set field singleKey multiKey;};

# -- Semantic grouped helpers ------------------------------------------------
mkMaturityGroup = {
  set,
  default ? "unknown",
}:
  mkEqQueries {
    inherit set default;
    field = "maturity";
  };

mkProtocolGroup = set:
  mkMemberQueries {
    inherit set;
    field = "protocol";
  };

mkCapabilityGroup = set: {
  acceleration = withFlag {
    inherit set;
    field = "acceleration";
  };
  compositing = withFlag {
    inherit set;
    field = "compositing";
  };
  remote = withFlag {
    inherit set;
    field = "remote";
  };
};

mkIndependenceQueries = set:
  mkBoolQueries {
    inherit set;
    field = "independent";
    trueKey = "independent";
    falseKey = "integrated";
  };

mkMaturityQueries = set: {
  stable = withEq {
    inherit set;
    field = "maturity";
    value = "stable";
  };
  young = withEq {
    inherit set;
```

```
    field = "maturity";
    value = "young";
  };
  legacy = withEq {
    inherit set;
    field = "maturity";
    value = "legacy";
  };
};

mkProtocolQueries = set: {
  onWayland = withMember {
    inherit set;
    field = "protocol";
    value = "wayland";
  };
  onKMS = withMember {
    inherit set;
    field = "protocol";
    value = "kms";
  };
  onTTY = withMember {
    inherit set;
    field = "protocol";
    value = "tty";
  };
  onXorg = withMember {
    inherit set;
    field = "protocol";
    value = "xorg";
  };
};

mkCapabilityQueries = set: {
  acceleration = withFlag {
    inherit set;
    field = "acceleration";
  };
  compositing = withFlag {
    inherit set;
    field = "compositing";
  };
  remote = withFlag {
    inherit set;
    field = "remote";
  };
};

mkConfigQueries = set: {
  configurable =
    filterAttrs
    (_: a: (a.config or null) != null)
    set;
};

#╔═══════════════════════════════════════════════════════════╗
#║ Core                                                      ║
#╚═══════════════════════════════════════════════════════════╝
filters = mkFilters {
  inherit all categories channels families;
  queried = {byCategory, ...}: let
    needsTerminal = withFlag {
      field = "needsTerminal";
      set = all;
    };
```

```
shell = let
  set = byCategory.shell;
  all = set;
in {
  inherit all;
  shells = let
    set = mapAttrs (_: val:
      val
      // {fileName = val.config.file or "none";}) (
      filterAttrs
      (_: a: (a.categories or []) == ["shell"])
      byCategory.shell
    );
    all = set;

    grouped = {
      byEngine = mkMemberQueries {
        inherit set;
        field = "engine";
      };
      byMaturity = mkMaturityGroup {inherit set;};
      byConfig = mkEqQueries {
        inherit set;
        field = "fileName";
      };
    };

    queried = let
      profileOnly = grouped.byFile.".profile" or {};
      configurable = removeAttrs all (attrNames profileOnly);
    in
      {
        inherit configurable;
        system = withFlag {
          inherit set;
          field = "system";
        };
        interactive = withFlag {
          inherit set;
          field = "interactive";
        };
      }
      // mkMaturityQueries set
      // mkBoolQueries {
        inherit set;
        field = "posix";
        trueKey = "posix";
        falseKey = "modern";
      };
  in {inherit all grouped queried;};

  prompts = let
    set = byCategory.prompt;
    all = set;

    grouped = {
      byShell = mkMemberQueries {
        inherit set;
        field = "shells";
      };
      byLanguage = mkEqQueries {
        inherit set;
        field = "language";
      };
      byMaturity = mkMaturityGroup {inherit set;};
```

```
    };

    queried =
      {}
      // mkConfigQueries set
      // mkMaturityQueries set
      // mkScopeQueries {
        inherit set;
        field = "shells";
        singleKey = "singleShell";
        multiKey = "multiShell";
      };
  in {inherit all grouped queried;};

  enhancements = let
    set = byCategory.enhancement;
    all = set;

    grouped = {
      byKind = mkEqQueries {
        inherit set;
        field = "kind";
      };
      byShell = mkMemberQueries {
        inherit set;
        field = "shells";
      };
      byEngine = mkMemberQueries {
        inherit set;
        field = "engine";
      };
      byMaturity = mkMaturityGroup {inherit set;};
    };

    queried =
      {
        history = withEq {
          inherit set;
          field = "kind";
          value = "history";
        };
        fuzzy = withEq {
          inherit set;
          field = "kind";
          value = "fuzzy";
        };
        navigation = withEq {
          inherit set;
          field = "kind";
          value = "navigation";
        };
      }
      // mkConfigQueries set
      // mkMaturityQueries set
      // mkScopeQueries {
        inherit set;
        field = "shells";
        singleKey = "singleShell";
        multiKey = "multiShell";
      };
  in {inherit all grouped queried;};

  lineEditors = let
    set = byCategory."line-editor";
    all = set;
    grouped = {
```

```
    grouped = {
      byEngine = mkMemberQueries {
        inherit set;
        field = "engine";
      };
      byShell = mkMemberQueries {
        inherit set;
        field = "shells";
      };
      byMaturity = mkMaturityGroup {inherit set;};
    };
    queried =
      {}
      // (mkConfigQueries set)
      // (mkMaturityQueries set)
      // (mkScopeQueries {
        inherit set;
        field = "shells";
        singleKey = "singleShell";
        multiKey = "multiShell";
      });
  in {inherit all grouped queried;};
};

interface = let
  set = byCategory.interface;
  all = set;

  compositors = let
    set = byCategory.compositor;
    all = set;
    grouped = {
      byProtocol = mkProtocolGroup set;
      byRole = mkEqQueries {
        inherit set;
        field = "role";
      };
      byMaturity = mkMaturityGroup {inherit set;};
    };
    queried =
      {}
      // (mkMaturityQueries set)
      // (mkProtocolQueries set)
      // (mkConfigQueries set);
  in {inherit all grouped queried;};

  environments = let
    set = byCategory.environment;
    all = set;

    grouped = {
      byCompositor = mkEqQueries {
        inherit set;
        field = "compositor";
      };
      byPanel = mkMemberQueries {
        inherit set;
        field = "panel";
      };
      byScope = mkEqQueries {
        inherit set;
        field = "scope";
      };
      byLayout = mkMemberQueries {
        inherit set;
        field = "layouts";
```

```
    };
    byProtocol = mkProtocolGroup set;
    byGreeter = mkMemberQueries {
      inherit set;
      field = "greeters";
    };
  };

  queried =
    {
      tiling = withMember {
        inherit set;
        field = "layouts";
        value = "tiling";
      };
      floating = withMember {
        inherit set;
        field = "layouts";
        value = "floating";
      };
      stacking = withMember {
        inherit set;
        field = "layouts";
        value = "stacking";
      };
    }
    // mkProtocolQueries set
    // mkEqQueries {
      inherit set;
      field = "scope";
    };
in {inherit all grouped queried;};

greeters = let
  set = byCategory.greeter;
  all = set;

  grouped = {
    byKind = mkEqQueries {
      inherit set;
      field = "kind";
    };
    byToolkit = mkEqQueries {
      inherit set;
      field = "toolkit";
    };
    byProtocol = mkProtocolGroup set;
  };

  queried =
    {}
    // mkIndependenceQueries set
    // mkMaturityQueries set
    // mkEqQueries {
      inherit set;
      field = "toolkit";
    }
    // mkEqQueries {
      inherit set;
      field = "kind";
    };
in {inherit all grouped queried;};

notifiers = let
  set = byCategory.notifier;
```

```
  all = set;

  grouped = {
    byMaturity = mkMaturityGroup {inherit set;};
    byProtocol = mkProtocolGroup set;
    byConfigLanguage = mkMemberQueries {
      inherit set;
      field = "config.lang";
    };
  };

  queried =
    {}
    // mkIndependenceQueries set
    // mkMaturityQueries set
    // mkProtocolQueries set
    // mkConfigQueries set;
in {inherit all grouped queried;};

panels = let
  set = byCategory.panel;
  all = set;
  grouped = {
    byToolkit = mkEqQueries {
      inherit set;
      field = "toolkit";
    };
    byMaturity = mkMaturityGroup {inherit set;};
    byProtocol = mkProtocolGroup set;
    byConfigLanguage = mkMemberQueries {
      inherit set;
      field = "config.lang";
    };
    byEngineLanguage = mkMemberQueries {
      inherit set;
      field = "engine";
    };
  };
  queried =
    {}
    // mkIndependenceQueries set
    // mkMaturityQueries set
    // mkProtocolQueries set
    // mkConfigQueries set
    // mkEqQueries {
      inherit set;
      field = "toolkit";
    };
in {inherit all grouped queried;};

protocols = let
  set = byCategory.protocol;
  all = set;
  grouped = {
    byCapability = mkCapabilityGroup set;
    bySurface = mkEqQueries {
      inherit set;
      field = "surface";
    };
    byMaturity = mkMaturityGroup {inherit set;};
  };
  queried =
    {}
    // mkCapabilityQueries set
    // mkMaturityQueries set
    // mkEqQueries {
```

```
            // mkEqQueries {
              inherit set;
              field = "surface";
            };
        in {inherit all grouped queried;};
      in {
        inherit
          all
          compositors
          environments
          greeters
          notifiers
          panels
          protocols
          ;
      };
    in {inherit needsTerminal shell interface;};
  };
in
  __exports.internal // {_rootAliases = __exports.external;}
```

Libraries/nix/applications/construction.nix

```
{_, ...}: let
  exports = {
    internal = {
      inherit
        mkFilters
        mkRegistry
        mkShellApp
        mkScriptWrapper
        mkScriptWrappers
        mkSubsystem
        ;
    };
    external = exports.internal;
  };

  inherit (_.attrsets.transformation) filterAttrs mapAttrs mapAttrsToList;
  inherit (_.attrsets.construction) genAttrs listToAttrs optionalAttrs;
  inherit (_.lists.access) head init last length;
  inherit (_.filesystem.access) readFile;
  inherit (_.filesystem.importers) importAllMerged;
  inherit (_.lists.selection) filter;
  inherit (_.lists.predicates) isIn;
  inherit (_.strings.construction) concatStringsSep indentedForError;
  inherit (_.strings.transformation) escapeShellArgs;
  inherit (_.types.predicates) isString isPath;

  normalizeOptional = val:
    if val == null || val == "" || val == "none"
    then null
    else val;

  mkFilters = {
    all,
    categories,
    channels ? {},
    families ? {},
    grouped ? {},
    queried ? (_: {}),
  }: let
    listCategories = indentedForError {
      title = "Valid Categories";
      items = categories.allValues;
```

```
  };
  byCategory = genAttrs categories.allValues ofCategory;
  byFamily =
    optionalAttrs
    (families?allValues)
    (
      genAttrs
      families.allValues
      (n:
        filterAttrs
        (_: a: (a.family or null) == n)
        all)
    );
  byChannel =
    optionalAttrs
    (channels?allValues)
    (
      genAttrs
      channels.allValues
      (n:
        filterAttrs
        (_: a: (a.channel or null) == n)
        all)
    );
  ofCategory = category:
    if !categories.validator.check category
    then
      throw
      "'${category}' is not a valid category. ${listCategories}"
    else
      filterAttrs
      (_: a: isIn category (a.categories or []))
      all;
in {
  inherit all;
  grouped =
    {inherit byCategory byChannel byFamily ofCategory;}
    // grouped;
  queried = queried {inherit byCategory byChannel byFamily;};
};

mkSubsystem = {
  path,
  categories,
  channels ? null,
  families ? null,
}: let
  all = mkRegistry {
    data = importAllMerged path {};
    inherit categories channels families;
  };
in {
  registry = all;
  filters = mkFilters {inherit all categories channels families;};
};

/**
Build a validated, normalized registry attrset from raw imported data.

# Arguments
- data:       imported attrset (e.g. from importAllMerged)
- categories: mkEnum result — required, non-nullable
- channels:   mkEnum result — optional, pass null to skip validation
- families:   mkEnum result — optional, pass null to skip validation
```

```
# Example
\```nix
all = mkRegistry {
  data       = _.filesystem.importers.importAllMerged ./.data {};
  inherit categories channels families;
};
\```
*/
mkRegistry = {
  data,
  categories,
  channels ? null,
  families ? null,
}: let
  listCategories = indentedForError {
    title = "Valid Categories";
    items = categories.allValues;
  };
  listChannels =
    if channels != null
    then
      indentedForError {
        title = "Valid Channels";
        items = channels.allValues;
      }
    else "";
  listFamilies =
    if families != null
    then
      indentedForError {
        title = "Valid Families";
        items = families.allValues;
      }
    else "";
in
  mapAttrs (name: app: let
    channel =
      if channels != null
      then normalizeOptional (app.channel or null)
      else null;
    family =
      if families != null
      then normalizeOptional (app.family  or null)
      else null;
    app' = app // {inherit channel family;};

    invalidCats = filter (c: !categories.validator.check c) app'.categories;
    catCount = length invalidCats;
    quotedCats = map (c: "'${c}'") invalidCats;
    humanJoin = items:
      if catCount == 1
      then head items
      else "${concatStringsSep ", " (init items)} and ${last items}";
  in
    if invalidCats != []
    then
      throw "${humanJoin quotedCats} ${
        if catCount == 1
        then "is an invalid category"
        else "are invalid categories"
      }. ${listCategories}"
    else if channels != null && !channels.validator.check app'.channel
    then throw "'${name}' has invalid channel '${toString app'.channel}'. ${listChannels}"
    else if families != null && !families.validator.check app'.family
    then throw "'${name}' has invalid family '${toString app'.family}'. ${listFamilies}"
    else app')
```

```
    else app )
  data;

/**
  mkShellApp - A helper function to create a shell script application with runtime dependencies
  and optional aliases.

  # Arguments
  - name (string):            The name of the application
  - inputs (list, optional):  List of packages to include in the runtime PATH (default: [])
  - command (string):         The shell script content to execute
  - prefix (string, optional): Prefix to add to command and alias names (default: "")
  - aliases (list, optional): List of alias specifications {name, description, prefix?}
  - description (string, optional): Description of the command for help text

  # Returns
  An attrset containing the main application and all its aliases

  # Example
\```nix
  mkShellApp {
    name = "dots";
    prefix = ".";
    inputs = with pkgs; [ rust-script ];
    command = ''
      exec rust-script "$@"
    '';
    description = "Main dotfiles CLI";
    aliases = [
      {name = "rebuild"; description = "Rebuild the system";}
      {name = "update"; description = "Update flake inputs";}
    ];
  }
\```
  Returns: { ".dots" = <derivation>; ".rebuild" = <derivation>; ".update" = <derivation>; }
*/
mkShellApp = {
  pkgs,
  /*
  The name of the script to write.

  Type: String
  */
  name,
  /*
  The shell script's text, not including a shebang.

  Type: String
  */
  command,
  /*
  Inputs to add to the shell script's `$PATH` at runtime.

  Type: [String|Derivation]
  */
  inputs ? [],
  /*
  Prefix to add to the command and alias names.

  Type: String
  */
  prefix ? "",
  /*
  List of aliases to create for this command.
  Each alias is an attrset with {name, description, prefix?}.
```

```
Type: [{name: String, description: String, prefix?: String}]
*/
aliases ? [],
/*
Optional description for the command (used in help text).

Type: String
*/
description ? null,
/*
Extra environment variables to set at runtime.

Type: AttrSet
*/
runtimeEnv ? null,
/*
`stdenv.mkDerivation`'s `meta` argument.

Type: AttrSet
*/
meta ? {},
/*
`stdenv.mkDerivation`'s `passthru` argument.

Type: AttrSet
*/
passthru ? {},
/*
The `checkPhase` to run. Defaults to `shellcheck` on supported
platforms and `bash -n`.

The script path will be given as `$target` in the `checkPhase`.

Type: String
*/
checkPhase ? null,
/*
Checks to exclude when running `shellcheck`, e.g. `[ "SC2016" ]`.

See <https://www.shellcheck.net/wiki/> for a list of checks.

Type: [String]
*/
excludeShellChecks ? [],
/*
Extra command-line flags to pass to ShellCheck.

Type: [String]
*/
extraShellCheckFlags ? [],
/*
Bash options to activate with `set -o` at the start of the script.

Defaults to `[ "errexit" "nounset" "pipefail" ]`.

Type: [String]
*/
bashOptions ? [
  "errexit"
  "nounset"
  "pipefail"
],
/*
Extra arguments to pass to `stdenv.mkDerivation`.
```

```
    :::note{.caution}
    Certain derivation attributes are used internally,
    overriding those could cause problems.
    :::

    Type: AttrSet
    */
    derivationArgs ? {},
    /*
    Whether to inherit the current `$PATH` in the script.

    Type: Bool
    */
    inheritPath ? true,
  }: let
    fullName = "${prefix}${name}";

    #> Create the main application
    mainApp = pkgs.writeShellApplication {
      name = fullName;
      inherit
        bashOptions
        checkPhase
        derivationArgs
        excludeShellChecks
        extraShellCheckFlags
        inheritPath
        runtimeEnv
        ;
      meta =
        meta
        // optionalAttrs (description != null) {
          inherit description;
        };
      passthru =
        passthru
        // {
          inherit aliases prefix;
          cmdDescription = description;
        };
      runtimeInputs = inputs;
      text = command;
    };

    #> Create alias applications
    aliasApps =
      map (aliasSpec: let
        aliasPrefix = aliasSpec.prefix or prefix;
        aliasName = "${aliasPrefix}${aliasSpec.name}";
      in {
        name = aliasName;
        value = pkgs.writeShellApplication {
          name = aliasName;
          runtimeInputs = [mainApp];
          text = ''exec ${fullName} ${aliasSpec.name} "$@"'';
          meta = {
            description = aliasSpec.description or "Alias for ${fullName} ${aliasSpec.name}";
          };
          passthru = {
            aliasOf = mainApp;
            aliasCmd = aliasSpec.name;
          };
        };
      })
      aliases;
  in
```

```
  #> Return attrset with main app and all aliases
  {${fullName} = mainApp;}
  // listToAttrs aliasApps;

/**
  mkScriptWrapper - Copies a POSIX shell script from the dotfiles tree into the nix
  store and wraps it in a named binary. The script is stored immutably at build time,
  making the resulting binary self-contained and reproducible across machines — while
  the source script remains a single canonical file usable anywhere POSIX is available.

  # Arguments
  - pkgs (AttrSet):           Nixpkgs instance
  - name (string):            Name of the resulting binary
  - script (path):            Path to the source shell script
  - extraArgs (list):         Extra arguments to prepend when invoking the script (default: [])

  # Returns
  A derivation providing a binary at `bin/<name>`

  # Example
\```nix
  mkScriptWrapper {
    inherit pkgs;
    name = "zen";
    script = tree.sh.local + "/packages/wrappers/zen.sh";
  }

  mkScriptWrapper {
    inherit pkgs;
    name = "feet-quake";
    script = tree.sh.local + "/packages/wrappers/feet.sh";
    extraArgs = ["--quake"];
  }
\```
*/
mkScriptWrapper = {
  pkgs,
  /*
  Name of the resulting binary.

  Type: String
  */
  name,
  /*
  Path to the source POSIX shell script in the dotfiles tree.
  The script is copied into the nix store at build time.

  Type: Path | String
  */
  script,
  /*
  Extra arguments to prepend when invoking the script.
  Useful for creating mode variants of the same script.

  Type: [String]
  */
  extraArgs ? [],
}: let
  inherit (pkgs) writeShellScript writeShellScriptBin;
  stored = writeShellScript "${name}.sh" (readFile script);
in
  writeShellScriptBin name ''
    exec ${stored} ${escapeShellArgs extraArgs} "$@"
  '';
```

```
  /**
    mkScriptWrappers - Batch-create script wrappers from an attrset of name → script path.
    All wrappers share the same pkgs instance.

    # Arguments
    - pkgs (AttrSet):   Nixpkgs instance
    - scripts (AttrSet): Mapping of binary name → script path (or { script, extraArgs } attrset)

    # Returns
    A list of derivations, suitable for use in `home.packages` or `environment.systemPackages`

    # Example
  \```nix
    mkScriptWrappers {
      inherit pkgs;
      scripts = {
        zen  = tree.sh.local + "/packages/wrappers/zen.sh";
        feet = tree.sh.local + "/packages/wrappers/feet.sh";
        # With extra args:
        feet-quake   = { script = tree.sh.local + "/packages/wrappers/feet.sh"; extraArgs = ["--quake"];   };
        feet-monitor = { script = tree.sh.local + "/packages/wrappers/feet.sh"; extraArgs = ["--monitor"]; };
      };
    }
  \```
  */
  mkScriptWrappers = {
    pkgs,
    scripts,
  }:
    mapAttrsToList (name: value:
      mkScriptWrapper (
        {inherit pkgs name;}
        // (
          if isPath value || isString value
          then {script = value;}
          else value # already an attrset with { script, extraArgs?, ... }
        )
      ))
    scripts;
in
  exports.internal // {_rootAliases = exports.external;}
```

Libraries/nix/applications/enums.nix

```
{_, ...}: let
  __exports = {
    internal = enums.static // enums.derived;
    external.applicationEnums = enums;
  };

  inherit (_.attrsets.access) attrValues;
  inherit (_.attrsets.transformation) mapAttrs;
  inherit (_.lists.access) head;
  inherit (_.lists.construction) mkEnum;
  inherit (_.types.predicates) isAttrs;
  inherit (_.applications.filters.queried) shell interface;

  isRegistryAttrset = tree:
    (tree != {})
    && (
      let
        firstVal = head (attrValues tree);
      in
        isAttrs firstVal && firstVal ? categories
```

```
    );

  toEnums = input:
    if isRegistryAttrset input
    then
      mkEnum {
        values = input;
        nullable = true;
      }
    else mapAttrs (_: subtree: toEnums subtree) input;

  enums = {
    static = {
      categories = mkEnum [
        #~@ Common
        "browser"
        "communication"
        "editor"
        "email-client"
        "file-manager"
        "game"
        "graphics"
        "launcher"
        "media"
        "messenger"
        "monitor"
        "office"
        "process"
        "system"
        "terminal"

        #~@ Interface
        "interface"
        "compositor"
        "environment"
        "greeter"
        "notifier"
        "panel"
        "protocol"

        #~@ Shell
        "shell"
        "prompt"
        "line-editor"
        "enhancement"
      ];
      channels = mkEnum {
        values = [
          "stable"
          "beta"
          "nightly"
          "insiders"
          "twilight"
          "esr"
          "legacy"
        ];
        nullable = true;
      };
      families = mkEnum {
        values = [
          "firefox"
          "chromium"
          "zen"
          "vscode"
          "emacs"
          "vim"
```

```
          "whatsapp"
        ];
        nullable = true;
      };
    };

    derived = {
      shells =
        toEnums shell
        // {
          queried =
            toEnums shell.queried
            // {
              system = mkEnum {
                values = shell.queried.system;
                nullable = false;
              };
            };
        };
      interface = toEnums interface;
    };
  };
in
  __exports.internal // {_rootAliases = __exports.external;}
```

Libraries/nix/applications/registry.nix

```
{_, ...}: let
  __exports = {
    internal = all;
    external.applicationRegistry = all;
  };

  inherit (_.applications.construction) mkRegistry;
  inherit (_.applications.enums) categories channels families;
  inherit (_.filesystem.importers) importAllMerged;

  all = mkRegistry {
    data = importAllMerged ./.data {};
    inherit categories channels families;
  };
in
  __exports.internal // {_rootAliases = __exports.external;}
```